

RansomShield — Aide-mémoire de soutenance

Discours complet — 10 min de présentation + 5 min de démo

Codjo Fréjus Loïc SANGRONIO — 24 juin 2026

À lire au calme avant la soutenance ; en séance, s'en servir comme filet de sécurité, pas comme texte à réciter mot à mot. Le numéro « Diapo N » correspond exactement au numéro affiché dans le panneau de diapositives d'Impress : si vous êtes sur la bonne note, vous êtes sur la bonne diapo.

Diapo 1/25 — OUVERTURE — 0 :00

Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, bonjour. Je vous remercie pour l'occasion qui m'est donnée de présenter mon travail portant sur la conception et la mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé : RansomShield. Cette présentation durera dix minutes, suivie d'une démonstration live de cinq minutes sur des machines virtuelles réelles. Je reste à votre disposition pour vos questions à la fin.

Diapo 2/25 — SOMMAIRE — 0 :30

Ma présentation s'articule en cinq parties : le contexte et la problématique, la revue de littérature, la conception de la solution, sa réalisation avec les résultats obtenus, puis la conclusion. Je terminerai par une courte démonstration du système en fonctionnement.

Diapo 4/25 — CONTEXTE & PROBLÉMATIQUE — 1 :00

Les organisations dépendent aujourd'hui fortement de leurs systèmes informatiques, ce qui les expose de plus en plus aux cyberattaques — en particulier au ransomware, ce logiciel malveillant qui chiffre les données puis exige une rançon. Selon l'ENISA, ces attaques ont augmenté de 73 % en 2023 ; sans outil dédié, il faut en moyenne 21 jours pour les détecter, pour un coût qui dépasse souvent le million de dollars. Le problème, c'est que les antivirus classiques, basés sur des signatures, ne suffisent plus face à des comportements inconnus. D'où la question centrale de ce travail : comment détecter et répondre à un comportement ransomware, dans un environnement contrôlé, sans déployer de vrai malware ?

Diapo 5/25 — OBJECTIFS — 2 :30

L'objectif général est de concevoir un système de détection et de réponse aux attaques ransomware, basé sur la surveillance comportementale des fichiers et des processus. Concrètement, il s'agit de surveiller les événements en temps réel, de calculer un score de risque, de générer automatiquement des alertes et des incidents, de proposer des actions de protection avec un contrôle humain systématique, de sécuriser la console par une double authentification, et de valider le tout sur des scénarios d'attaque réalistes.

Diapo 6/25 — ORGANISATION DU MÉMOIRE — 3 :15

Le mémoire suit le canevas IFRI en trois chapitres : la revue de littérature, l'analyse et la conception du système, puis sa réalisation et ses résultats. Voyons d'abord les fondements théoriques.

Diapo 8/25 — DÉFINITION & COMPORTEMENTS — 3 :50

Selon le NIST, un ransomware est un logiciel malveillant qui bloque l'accès aux données, le plus souvent par chiffrement, pour ensuite réclamer une rançon. Ce qui compte pour nous, c'est que ce type d'attaque laisse des signes avant que les dégâts ne soient irréversibles : renommage massif de fichiers, apparition d'extensions suspectes comme `.locked`, création d'une note de rançon, suppression des sauvegardes système, ou encore utilisation détournée d'outils légitimes. Ce sont ces signaux qui constituent la base de notre détection comportementale.

Diapo 9/25 — SOLUTIONS EXISTANTES — 5 :15

Les solutions existantes ont chacune leurs limites. Les antivirus classiques détectent par signatures, donc ils passent à côté des nouvelles variantes. Les SIEM collectent des journaux mais sans corrélation comportementale spécifique. Les EDR commerciaux, comme CrowdStrike ou SentinelOne, sont efficaces mais hors de portée pour un contexte académique. C'est dans cet espace que se positionne RansomShield : une approche comportementale, open-source, avec un contrôle humain garanti à chaque étape. Voyons maintenant comment ce système a été conçu.

Diapo 11/25 — ARCHITECTURE — 6 :15

L'architecture repose sur un flux simple. Les agents Python, installés sur les machines surveillées, envoient les événements à l'API Laravel via une clé propre à chaque agent. Le moteur de détection analyse ces événements, calcule un score de risque, et déclenche une alerte puis un incident si le seuil est dépassé. L'opérateur consulte la console SOC et valide chaque action avant son exécution : c'est notre règle d'or, aucune action automatique sans validation humaine.

Diapo 12/25 — LES 4 COMPOSANTS — 7 :15

Le système repose sur quatre composants. L'agent Python surveille les fichiers en temps réel et conserve une file locale pour ne rien perdre en cas de coupure réseau. La console SOC centralise le suivi : alertes, incidents, et validation des actions. La découverte réseau scanne automatiquement les sous-réseaux, mais aucun agent ne peut s'inscrire sans validation manuelle de l'administrateur. Enfin, le module de simulation permet de rejouer des scénarios d'attaque réalistes, jusqu'à vingt-deux événements, sans jamais toucher à un vrai malware.

Diapo 13/25 — DIAGRAMME UML — 8 :45

Le système distingue deux acteurs. L'administrateur gère les agents, configure le système et lance les simulations. L'analyste SOC, lui, se concentre sur le traitement opérationnel : il visualise les alertes et les incidents, approuve ou rejette les actions de protection, et consulte la timeline d'audit — mais n'a pas accès à la configuration.

Diapo 14/25 — MODÈLE DE DONNÉES — 9 :45

Le modèle de données repose sur huit tables MySQL. Le pipeline se lit de gauche à droite : un agent génère des événements, qui déclenchent des alertes, qui créent des incidents, qui donnent lieu à des actions de protection — le tout tracé dans un journal d'audit. Chaque action est horodatée avec l'identité de l'opérateur qui l'a validée. Passons maintenant à la réalisation concrète.

Diapo 16/25 — CHOIX TECHNIQUES — 11 :00

Côté technique, le backend repose sur Laravel 11 et PHP 8.3, avec une base MySQL et une API sécurisée par une clé propre à chaque agent. L'agent de surveillance est écrit en Python avec la bibliothèque Watchdog, et conserve ses événements dans une file SQLite locale. Le frontend utilise Chart.js en local, sans dépendance externe. Les tests ont été menés sur trois machines virtuelles KVM, avec une authentification à double facteur.

Diapo 17/25 — AGENT & MOTEUR — 12 :15

Au démarrage, l'agent s'enrôle via un token à usage unique qui lui donne une clé API permanente, puis il surveille les dossiers avec Watchdog et envoie un signal de présence toutes les trente secondes. Le moteur de détection calcule un score cumulatif configurable. Par exemple : un fichier renommé en `.locked` vaut 80 points, une note de rançon 55 points ; le total de 135 déclenche immédiatement une alerte critique et la création d'un incident. Tout ce processus prend moins de cinq secondes.

Diapo 18/25 — CONSOLE SOC — 13 :30

La console SOC centralise dix pages fonctionnelles, avec un tableau de bord mis à jour automatiquement toutes les trente secondes. Le point important, c'est que toute action de protection passe par une file d'approbation : l'analyste doit explicitement valider ou rejeter chaque action avant qu'elle ne s'exécute. Les notifications partent simultanément par quatre canaux : le web, l'e-mail, le son, et un webhook. Vous verrez tout cela dans la démonstration.

Diapo 19/25 — TESTS & RÉSULTATS — 14 :45

Nous avons validé le système sur cinq scénarios d'attaque, de sept à vingt-deux événements. Tous ont été détectés en moins de cinq secondes, sans perte d'événement, avec un pipeline complet validé de bout en bout, de l'événement jusqu'à la timeline. Je précise honnêtement une limite : les tests ont été conduits uniquement sur Linux, et un attaquant qui ralentirait son rythme resterait moins visible. Passons maintenant à la démonstration du système.

Diapo 20/25 — DÉMO LIVE — 15 :00, checklist 5 min, hors chrono présentation

1. Ouvrir `http://10.20.0.1` (console SOC)
2. Se connecter : identifiants + **code 2FA**
3. Vérifier : 3 agents en ligne (heartbeat vert)
4. **Agents** → VM-02 → **Lancer simulation** → "Kill chain complète" (22 évts)
5. **Dashboard** : le score monte en temps réel
6. **Alertes** : niveau Critique + signaux détectés
7. **Incidents** : ouvrir l'incident auto-créé
8. **File d'approbation** : approuver l'isolation réseau
9. **Timeline** : tout est horodaté et tracé

Rester calme, ne pas se précipiter — chaque étape a le temps qu'il faut.

En résumé, RansomShield est une solution fonctionnelle qui couvre tout le cycle de traitement d'un incident : collecte, analyse, alerte, réponse et audit. Elle repose sur un agent multi-plateforme, une console complète, un moteur de détection réactif, et un contrôle humain à chaque étape. Ce travail a ses limites : il a été testé dans un environnement contrôlé, avec des réponses en partie simulées. Les perspectives sont claires : ajouter le chiffrement HTTPS, des rôles distincts pour les utilisateurs, puis à terme du machine learning pour affiner la détection, et une intégration avec les outils SIEM du marché. Je vous remercie pour votre attention et reste à votre disposition pour vos questions.

- **Pourquoi Python et pas C/Go ?** → Watchdog exploite inotify nativement, CPU minimal, portable Linux/Windows/macOS
 - **Faux positifs ?** → seuils configurables, **opérateur humain** valide ou rejette chaque action
 - **Pourquoi pas HTTPS ?** → LAN contrôlé ; c'est la 1^{re} perspective pour la prod
 - **Différence avec un antivirus ?** → comportement, pas signature ; fonctionne sur variantes inconnues
 - **Testé sur Windows ?** → Non, limite assumée : Linux uniquement
- Garder les **annexes** prêtes (file d'approbation, captures d'écran).*